

VCR: Fast Private Set Intersection with Improved VOLE and CRT-Batching

Weizhan Jing¹ Xiaojun Chen¹ Xudong Chen¹ Ye Dong² Yaxi Yang³ and Qiang Liu¹

¹ Institute of Information Engineering, Chinese Academy of Sciences

² School of Computing, National University of Singapore

³ iTrust, Singapore University of Technology and Design

{jingweizhan, chenxiaojun, chenxudong, liuqiang2022}@iie.ac.cn,
dongye@nus.edu.sg, yxyangjnu@gmail.com

Abstract. Private set intersection (PSI) allows two participants to compute the intersection of their private sets without revealing any additional information beyond the intersection itself. It is known that oblivious linear evaluation (OLE) can be used to construct the online efficient PSI protocol (Kerschbaum *et al.*, NDSS’23). However, oblivious transfer (OT) and fully homomorphic encryption (FHE)-based offline OLE generation are expensive, and the online computational complexity is super-linear and still a heavy burden for large-scale sets.

In this paper, we propose VCR, an efficient PSI protocol from vector OLE (VOLE) with the offline-online paradigm. Concretely, we first propose the batched short VOLE protocol to reduce offline overhead for generating VOLE tuples. Experiments demonstrate that VCR outperforms prior art. Then, we design a batched private membership test protocol from pre-computed VOLE to accelerate the online computation. Compared to the previous work of Kerschbaum *et al.* (NDSS’23), we reduce the total communication costs (resp. running time) by $341\times$ and $9.1\times$ (resp. $6.5\times$ and $2.5\times$) on average for OT and FHE-based protocols.

Keywords: Private Set Intersection · Vector Oblivious Linear Evaluation · Chinese Remainder Theorem

1 Introduction

Private set intersection (PSI) is a cryptographic primitive that enables two parties (i.e., P_0 who has a set X and P_1 who has the set Y) to securely compute the intersection $X \cap Y$ without revealing additional information about the sets. It has motivated many applications (such as private contact discovery [14, 17, 19] and DNA testing [13]), in which blacklist matching [27, 8] is a significant one to the risk management. The service provider can secretly detect potentially malicious behavior in login or transaction requests by matching the IP or user ID with the blacklist. Then, measurements can be taken to reduce risk and minimize losses.

Over the past several years, PSI has been extensively studied and many efficient protocols have been proposed [7, 26, 25, 20, 23, 9, 12, 30]. However, these

protocols still fall short of meeting the real-time requirements of practical applications. For example, blacklist matching must be performed with low latency. Otherwise, it impairs the user experience and system performance. To meet the above requirements, some works [28, 18] adopt the offline-online paradigm to enable a faster online phase. And the work of [18] achieves the most efficient online phase by leveraging precomputed oblivious linear evaluation (OLE) tuples.

1.1 Previous OLE-based PSI

We provide an overview of the state-of-the-art (SOTA) OLE-based PSI [18] as follows. The core idea of [18] is bin-wise matching. In the offline phase, they first profile the set X , held by P_0 , into a cuckoo hash table and insert the set Y , held by P_1 , into a two-dimensional simple hash table using the same hash functions. Both the cuckoo hash table and simple hash table contain $\alpha = \mathcal{O}(n)$ bins, where n is the set size. For each item x in the cuckoo hash table, one only needs to compare it with the items $y_1, y_2, \dots, y_\beta$ in the simple hash table at the same bin location. Here, $\beta = \mathcal{O}(\log n / \log \log n)$ is the maximum bin load of the simple hash table. To facilitate this bin-wise comparison, the protocol generates β OLE tuples for each of the α bins ($\alpha \cdot \beta$ OLE tuples in total). Each tuple satisfies

$$r_{i,j}^0 \cdot r_{i,j}^1 = s_i^0 + s_{i,j}^1 \quad i \in [\alpha], j \in [\beta]$$

P_0 and P_1 hold $r_{i,j}^0, s_i^0 \in \mathbb{F}$ and $r_{i,j}^1 \in \mathbb{F} \setminus \{0\}, s_{i,j}^1 \in \mathbb{F}$, respectively.

In the online phase, P_0 computes $u_i = s_i^0 - x_i$ for i -th bin and sends it to P_1 , who evaluates $w_{i,j} = (u_i + y_{i,j} + s_{i,j}^1) / r_{i,j}^1, j \in [\beta]$ and returns the results back. After receiving $w_{i,j}, j \in [\beta]$, P_0 verifies whether $\exists j \in [\beta]$ s.t. $w_{i,j} = r_{i,j}^0$ and outputs the corresponding item x_i as an intersection item if it is true.

For the offline phase, the pure cryptographic-based method, i.e., fully homomorphic encryption (FHE) or oblivious transfer (OT), is the most commonly used for OLE generation. However, those methods take $\mathcal{O}(\alpha \cdot \beta) = \mathcal{O}(n \log n / \log \log n)$ communication complexity, incurring excessive overhead. As reported in [18], the FHE-based (resp. OT-based) OLE generation for set size $n = 2^{20}$ takes more than 482 MB (resp. 23896 MB) in communication and 14.7 seconds (resp. 44.7 seconds) in computation.

1.2 Motivation

Vector OLE (VOLE) [1] is the vectorized variant of OLE. More concretely, the VOLE enables one party, P_1 , gets two random vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}^\beta$ while another party, P_0 , gets a random value $\Delta \in \mathbb{F}$ and a vector $\mathbf{c}$ such that $\mathbf{c} = \Delta \cdot \mathbf{a} + \mathbf{b}$. Recent works [4, 31, 33, 29] demonstrate that the long VOLE instances can be generated more efficiently than the OLE from the learning parity with noise (LPN) assumption. This observation makes it natural to consider replacing β OLE instances with a single VOLE instance per bin in the OLE-based PSI protocol [18]. However, directly applying VOLE in place of OLE in the PSI protocol of [18] introduces several new challenges that must be carefully addressed.

- i) **Lower offline efficiency:** To replace the OLE tuple, the offline phase should independently generate the VOLE tuple with the length $\beta = \mathcal{O}(\log n / \log \log n)$ for each of α hash bins. According to [31, 33, 29], when the length of a VOLE tuple is smaller than 2^{10} , the efficiency of LPN-based VOLE tuple generation is lower than that of FHE-based (or OT-based) methods. For example, for the input $n = 2^{20}$, directly generating VOLE tuples to replace the OLE tuple takes more than 23970 seconds in computation and 516369 MB in communication, which is significantly larger than prior work [18]. Our first challenge thus is to generate a sufficient number of short VOLE tuples efficiently.
- ii) **Incompatible and super-linear online phase:** Since the difference of VOLE tuples ($\mathbf{c}[j] = \Delta \cdot \mathbf{a}[j] + \mathbf{b}[j], j \in [\beta]$) and the OLE tuples used in [18] ($r_j^0 \cdot r_j^1 = s^0 + s_j^1, j \in [\beta]$), the OLE-based PSI protocol of [18] is incompatible with the VOLE tuples. Therefore, a new VOLE-based protocol is required. Besides, with the hashing optimization, the work of [18] needs to compare one item x of P_0 with β items $y_1, \dots, y_\beta$ of P_1 for each of the α hash bins. In other words, the online computation complexity of the previous PSI protocol [18] is $\mathcal{O}(\alpha\beta) = \mathcal{O}(n \log n / \log \log n)$, which is super-linear with the set size and still a heavy burden for large-scale sets. Therefore, the second challenge is to construct a more efficient PSI protocol from the pre-computed VOLE.

1.3 Our Techniques

To tackle the above challenges, we propose VCR, a more efficient PSI, by two novel subprotocols: batched short VOLE (bsVOLE) and batched private membership test (bPMT). Following the previous work [18], VCR contains the offline VOLE generation phase and the online intersection computation phase.

In the offline phase, we propose bsVOLE that can generate a sufficient number of short and independent VOLE tuples. This approach first produces VOLE tuples column-wise and then converts them into independent row-wise VOLE tuples via remasking. Specifically, we first propose the fixed VOLE protocol, which could generate m VOLE instances ($\mathbf{c}'_j = \Delta'_j \cdot \mathbf{a}' + \mathbf{b}'_j, j \in [m]$) with the same vector $\mathbf{a}'$. After that, we let P_0 and P_1 perform the fixed VOLE protocol for the whole hash table (by column) to get β VOLE tuples $\mathbf{c}'_j = \Delta'_j \cdot \mathbf{a}' + \mathbf{b}'_j, j \in [\beta]$, where $\mathbf{a}', \mathbf{b}'_j \in \mathbb{F}^\alpha, \Delta'_j \in \mathbb{F}$. Then, we slice the above tuples by row. For i -th row, we have $\mathbf{c}'_j[i] = \Delta'_j \cdot \mathbf{a}'[i] + \mathbf{b}'_j[i], j \in [\beta]$. While this has been consistent with the target VOLE in form, the reuse of the Δ'_j between multiple rows makes the multiple VOLE tuples correlated. To tackle this, we use α OLE tuples to remask the Δ'_j of the sliced tuples and obtain independent VOLE tuples $\mathbf{c}_i = \Delta_i \cdot \mathbf{a}_i + \mathbf{b}_i, i \in [\alpha]$, where $\mathbf{a}_i, \mathbf{b}_i \in \mathbb{F}^\beta, \Delta_i \in \mathbb{F}$. By leveraging the fixed VOLE to operate over long vectors ($\alpha \gg \beta$), we avoid the inefficiency of directly generating a large number of short VOLE, thereby achieving performance improvements.

In the online phase, we propose bPMT, which enables P_0 to check whether an item x is in an input set Y of P_1 in a batched manner, from VOLE. The bPMT requires only lightweight plaintext field operations. First, we design a PMT protocol from the pre-computed VOLE. Next, we apply a Chinese Remainder Theorem

(CRT)-based batching approach to improve the online computational efficiency. However, using CRT-based batching directly increases communication costs. For example, when packing β items with ℓ -bit length into a big number by CRT, the length of the result is $(\ell + 1)\beta$, incurring β bits growth. To address this, we propose an optimized CRT-based batching to eliminate the overhead. Finally, by combining the VOLE-based PMT and optimized CRT-based batching, bPMT reduces the online computational cost without communication growth.

Furthermore, these subprotocols, bPMT and bsVOLE, may be of independent interest in their own right. For example, the bPMT could be used to construct the private set operation protocols [10]. The bsVOLE somewhat solves the inefficiency of generating short VOLE tuples from the LPN assumptions [31].

1.4 Contributions

In summary, our contribution is formally described as follows:

- **Offline Efficient bsVOLE:** We propose a bsVOLE protocol to improve the offline efficiency of VOLE generation. Compared to the pure cryptographic (FHE or OT)-based method of [18] (resp. directly performing the LPN-based method), we reduce the communication complexity of offline tuple generation from $\mathcal{O}(n \log n / \log \log n)$ (resp. $\mathcal{O}(n \log n)$) to $\mathcal{O}(n)$.
- **Online bPMT from VOLE:** We present the bPMT protocol, which performs the private membership test using pre-computed VOLE tuples and without any cryptographic operations in the online phase. To accelerate the computation, bPMT employs an optimized CRT-based batching technique that maintains constant communication overhead.
- **Implementation & Evaluation:** We implement VCR and compare it with the SOTA protocols. For the online phase, VCR is $1.9\text{-}2.6\times$ and $5.5\text{-}9.4\times$ faster than two SOTA PSI protocols [18] and [28], respectively. For the offline phase, we improve the running time up to $4.6\times$ (resp. $14.5\times$) and decrease communication overhead by an average of $22.3\times$ (resp. $886.5\times$) compared to the FHE (resp. OT)-based protocol of [18].

2 Preliminary

2.1 Notation

X, Y are the input sets with size n . And x_i is the i -th item of set X . Let $\mathbb{F}$ be some finite field and $\mathbb{F}^* = \mathbb{F} \setminus \{0\}$. $r \leftarrow \mathbb{F}$ means that r is assigned a uniformly random element from $\mathbb{F}$. $\mathbf{a}$ denotes a vector. For vector array, we use $\mathbf{a}_i$ to denote the i -th vector and $\mathbf{a}_i[j]$ for j -th item of vector $\mathbf{a}_i$. $[n] = \{1, 2, \dots, n\}$. The number of hash functions is h . The packaged results of the batching technique are labeled with the capital letter P as a corner marker. α denotes the length of the hash table and β denotes the maximum load of each bin in the hash table. $[\![\cdot]\!]$ indicates the ciphertext. κ and λ are the computational and statistical security parameters, respectively.

2.2 Vector OLE

The OLE, introduced by Applebaum *et al.* [1], is a secure two-party protocol that allows one party to learn a linear combination of several elements held by another party. A useful extension of OLE is vector OLE (VOLE) [4, 31, 33], i.e., one party obtains two random vectors $\mathbf{a}, \mathbf{b}$, while the other party obtains a random value Δ and a vector $\mathbf{c} = \Delta \cdot \mathbf{a} + \mathbf{b}$. In this work, we focus on the pseudorandom variant of VOLE.

Definition 1 (Pseudorandom VOLE). *A pseudorandom VOLE consists of the following two algorithms:*

- $(seed_0, seed_1) \leftarrow \text{VOLE.Setup}(1^\lambda, n, \mathbb{F}, \Delta)$: Given vector length n , field $\mathbb{F}$ and value Δ , this algorithm outputs a pair of seeds $(seed_0, seed_1)$.
- $\text{VOLE.Expand}(r, seed_r)$: If $r = 1$, output $(\mathbf{a}, \mathbf{b}) \in \mathbb{F}^n \times \mathbb{F}^n$, else if $r = 0$, then output $\mathbf{c} \in \mathbb{F}^n$.

The correctness of the protocol guarantees that $\mathbf{c} = \Delta \cdot \mathbf{a} + \mathbf{b}$. The security properties require that for any probabilistic polynomial-time (PPT) algorithm $\mathcal{A}$, it holds that

$$\begin{aligned} & \left| \Pr[r = r' | r' \leftarrow \mathcal{A}(seed_1), (seed_0, seed_1) \leftarrow \text{VOLE.Setup}(1^\lambda, n, \mathbb{F}, \Delta_r), \right. \\ & \quad \left. (\mathbb{F}, n, \Delta_0, \Delta_1) \leftarrow \mathcal{A}(1^\lambda)] - 1/2 \right| \leq \text{negl}. \\ & \left| \Pr[r = r' | r' \leftarrow \mathcal{A}(\mathbf{a}_r, \mathbf{b}_r, seed_0), (seed_0, seed_1) \leftarrow \text{VOLE.Setup}(1^\lambda, n, \mathbb{F}, \Delta), \right. \\ & \quad \left. (\mathbb{F}, n, \Delta) \leftarrow \mathcal{A}(1^\lambda), (\mathbf{a}_1, \mathbf{b}_1) \leftarrow \text{VOLE.Expand}(1, seed_1), \mathbf{a}_0 \leftarrow \mathbb{F}^n, \right. \\ & \quad \left. \mathbf{c} \leftarrow \text{VOLE.Expand}(0, seed_0), \mathbf{b}_0 = \mathbf{c} - \mathbf{a}_0 \Delta] - 1/2 \right| \leq \text{negl}. \end{aligned}$$

2.3 Function Secret Sharing

Multi-point function secret sharing (MPFSS) [5, 6] is a primitive that allows a dealer to distribute the shares of a function $f_{S,\mathbf{e}} : [n] \rightarrow \mathbb{F}$ between two parties, and each party can use its share to locally compute the share of $f_{S,\mathbf{e}}(s)$ for every input s . We describe the ideal functionality of MPFSS in Figure 1. In this work, we rely on the MPFSS of [29] to construct the VOLE.

Functionality $\mathcal{F}_{\text{MPFSS}}$
Parameters: A multi-point function $f_{S,\mathbf{e}} : [n] \rightarrow \mathbb{F}$, where $S = \{s_1, \dots, s_t\}$ is a subset of $[n]$, $\mathbf{e} = [e_1, \dots, e_t] \in \mathbb{F}^t$, $f_{S,\mathbf{e}}(s) = e_i$ for $s \in S$, and $f_{S,\mathbf{e}}(s) = 0$ for any $s \in [n] \setminus S$. Key Generation ($\text{MPFSS.Gen}(1^\lambda, f_{S,\mathbf{e}})$): Given the security parameter λ and a description of $f_{S,\mathbf{e}} : [n] \rightarrow \mathbb{F}$, outputs a pair of keys (K_0, K_1). Expansion ($\text{MPFSS.Eval}(r, K_r, s)$): Given an evaluation key K_r for $r \in \{0, 1\}$ and an input s, output an additive share z_r of the value $f_{S,\mathbf{e}}(s)$ ($z_0 + z_1 = f_{S,\mathbf{e}}(s)$).

Fig. 1. Ideal functionality $\mathcal{F}_{\text{MPFSS}}$ of multi-point function secret sharing

CRT-based batching

- **Batch.Pack**($m_1, \dots, m_\gamma$): Given the messages $m_1, \dots, m_\gamma$, outputs primes $q_i \geq m_i, i \in [\gamma]$ and a number S^P such that $m_i = S^P \pmod{q_i}$.
- **Batch.Unpack**($S^P, q_1, \dots, q_\gamma$): Given a number S^P , outputs items $m_1, \dots, m_\gamma$ such that $m_i = S^P \pmod{q_i}, i \in [\gamma]$.
- **Batch.Eval**($(S_1^P, \dots, S_k^P), f$): Given an arithmetic circuit f with k input wires, and inputs $S_1^P, \dots, S_k^P$ with $S_i^P = \text{Batch.Pack}(m_{i,1}, \dots, m_{i,\gamma}), i \in [k]$, outputs a number $S^{P,i} = \text{Batch.Pack}(f(m_{i,1}, \dots, m_{i,\gamma}), \dots, f(m_{i,1}, \dots, m_{i,\gamma}))$.

Fig. 2. Three algorithms of CRT-based batching**2.4 CRT-based batching**

The Chinese Remainder Theorem (CRT)-based batching, shown in Figure 2, allows multiple data operations to be performed simultaneously, improving the computational efficiency of operations. It bears emphasizing that CRT-based batching only supports addition and scale multiplication operations.

2.5 Hashing Technique

Cuckoo hashing. Cuckoo hashing [22] can be seen as a way to solve the collision of hashing. It involves utilizing multiple hash functions, denoted as $H_1, \dots, H_h$, to distribute n items into a hash table CH . Each bin in CH can only hold one item. When inserting an item e , cuckoo hashing randomly selects a hash function H_i and maps e into the bin indexed by $H_i(e)$. If the target bin is occupied by e' , e' is evicted and replaced with e . Then, e' is rehashed using a different hash function until an empty bin is found. If insertion fails after several attempts, the evicted item is stored in a buffer pool known as the stash. The work of [18] demonstrates that when employing $h = 3$ hash functions and setting the hash table size to $\alpha = 1.27n$, the stash size can be effectively reduced to zero.

Permutation-Based Hashing. The bit-length of the item stored in the hash table can be reduced by the permutation-based hashing technique [2, 18]. When inserting a ℓ -bit item e into the hash table by h hash functions, the item is represented as $e_L || e_R$, where the length of e_L is $\log \alpha$. Then, the hash function $H_i, i \in [h]$ maps the item e to the bin $H_i(e_R) \oplus e_L$, and the value $i || e_R$ with $\ell - \log \alpha + \lceil \log h \rceil$ bits is stored.

2.6 Security Model

As in previous protocols [20, 18], we use the standard notion of security in the presence of semi-honest adversaries. Let Π be a two-party protocol for computing the functionality $f(X, Y)$, where X and Y are the input of P_0 and P_1 , and $\text{output}(X, Y)$ be the output of both parties. Define $\text{View}_0^\Pi(X, Y)$ and $\text{View}_1^\Pi(X, Y)$ be the views of P_0 and P_1 .

Definition 2 (Semi-Honest Security). *Two-party protocol Π securely realize function f against semi-honest adversaries if for every PPT adversary $\mathcal{A}$ there exists PPT simulators Sim_0 and Sim_1 such that for all inputs X and Y ,*

$$\begin{aligned} \{\text{View}_0^\Pi(X, Y), \text{output}(X, Y)\} &\cong_c \{Sim_0(X, f(X, Y)), f(X, Y)\} \\ \{\text{View}_1^\Pi(X, Y), \text{output}(X, Y)\} &\cong_c \{Sim_1(Y, f(X, Y)), f(X, Y)\} \end{aligned}$$

where $\cong_c$ represents computational indistinguishability.

3 Batched Short VOLE for Offline Phase

In this section, we propose batched short VOLE (bsVOLE), a new primitive for two-party VOLE generation that leverages MPFSS to generate a sufficient number of short and independent VOLE tuples. Specifically, it aims to generate the correlated outputs $\mathbf{c}_i = \Delta_i \cdot \mathbf{a}_i + \mathbf{b}_i, i \in [n]$, wherein P_0 gets $\mathbf{c}_i \in \mathbb{F}^\omega, \Delta_i \in \mathbb{F}$ and P_1 gets $\mathbf{a}_i \in \{\mathbb{F}^*\}^\omega, \mathbf{b}_i \in \mathbb{F}^\omega$ ($\omega \ll n$). As a building block, we first present the fixed VOLE protocol in section 3.1. Then, we construct the bsVOLE from the fixed VOLE in Section 3.2.

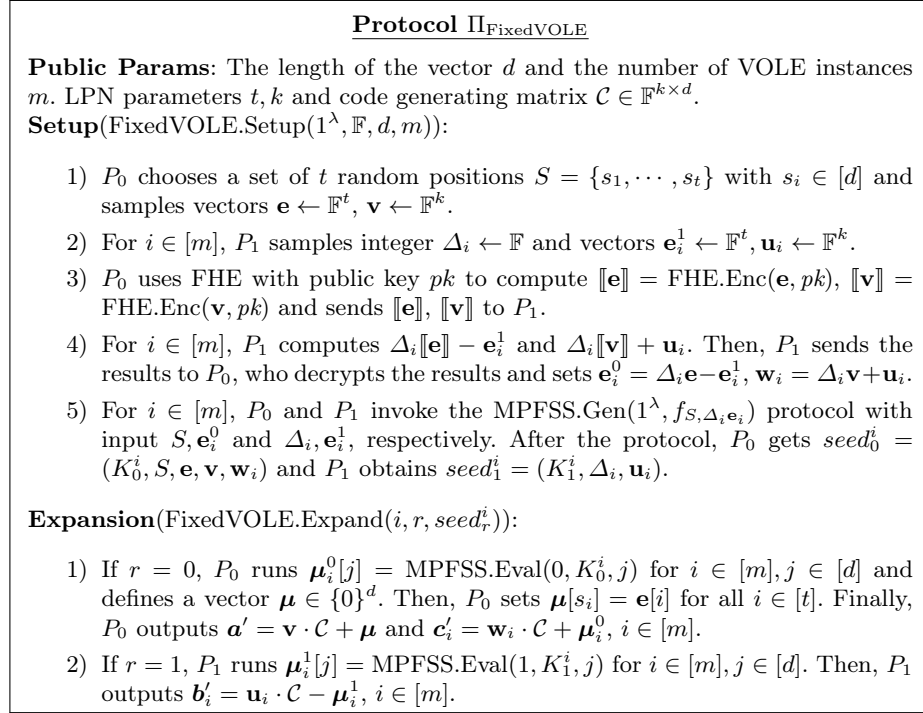
3.1 Fixed VOLE

The ideal functionality $\mathcal{F}_{\text{FixedVOLE}}$ of fixed VOLE is shown in Figure 3. Different from the standard VOLE, $\mathcal{F}_{\text{FixedVOLE}}$ outputs multiple VOLE tuples $\mathbf{c}'_i = \Delta'_i \cdot \mathbf{a}' + \mathbf{b}'_i, i \in [m]$, wherein P_0 holds $\mathbf{c}'_i, \mathbf{a}'$ and P_1 holds $\Delta'_i, \mathbf{b}'_i$, with the same $\mathbf{a}'$.

Functionality $\mathcal{F}_{\text{FixedVOLE}}$	
Public Params: Two parties, P_0 and P_1 . A finite field $\mathbb{F}$, the length of VOLE tuples d , the number of VOLE instances m .	
Functionality: Upon receiving (P_0, sid) from P_0 and (P_1, sid) from P_1 ,	
1)	$\mathcal{F}_{\text{FixedVOLE}}$ samples random vectors $\{\mathbf{b}'_i\}_{i \in [m]} \leftarrow \mathbb{F}^{d \times m}, \mathbf{a}' \leftarrow \mathbb{F}^d$ and integers $\{\Delta'_i\}_{i \in [m]} \leftarrow \mathbb{F}^m$. Then, it computes $\mathbf{c}'_i = \Delta'_i \cdot \mathbf{a}' + \mathbf{b}'_i$ for $i \in [m]$.
2)	$\mathcal{F}_{\text{FixedVOLE}}$ sends $\{\Delta'_i\}_{i \in [m]}, \{\mathbf{b}'_i\}_{i \in [m]}$ to P_1 and $\mathbf{a}', \{\mathbf{c}'_i\}_{i \in [m]}$ to P_0 .

Fig. 3. Ideal functionality $\mathcal{F}_{\text{FixedVOLE}}$ of fixed VOLE

The fixed VOLE follows the blueprint of [31, 33, 29, 4], which uses LPN to convert the MPFSS output into the VOLE tuples locally. In the LPN-based VOLE generation [31, 3, 4, 33], the vector $\mathbf{a}'$ is generated by $\mathbf{a}' = \mathbf{v} \cdot \mathcal{C} + \boldsymbol{\mu}$, where $\mathcal{C}$ is a public matrix, $\mathbf{v}$ is a vector sampled by P_0 , and $\boldsymbol{\mu}$ is a noise vector generated from MPFSS. Since the vector $\mathbf{v}$ is freely chosen by P_0 and can be identical in multiple VOLE generations easily, the heart of the fixed VOLE is obtaining the identical noise vector $\boldsymbol{\mu}$ in different MPFSS protocols.

Fig. 4. Fixed VOLE protocol $\Pi_{\text{FixedVOLE}}$

Our key insight is that the vector $\boldsymbol{\mu}$ cannot be chosen but can be identical in multiple MPFSS. We observe that the vector $\boldsymbol{\mu}$ is only related to the inputs of P_0 . In other words, P_0 can always obtain the identical vector $\boldsymbol{\mu}$ with the same input, whatever the P_1 inputs. Consequently, the same vector $\mathbf{a}'$ can be obtained by P_0 with the same inputs in multiple MPFSS protocols.

Upon the above observation, we propose the fixed VOLE protocol $\pi_{\text{FixedVOLE}}$ in Figure 4. Theorem 1 captures the correctness and security of $\Pi_{\text{FixedVOLE}}$ and the full proof is given in Appendix A.1.

Theorem 1 (Security). *If the LPN assumption holds and the FHE scheme is IND-CPA secure, the protocol $\Pi_{\text{FixedVOLE}}$ of Figure 4 correctly and securely realizes the functionality $\mathcal{F}_{\text{FixedVOLE}}$ against semi-honest adversaries in the random oracle, MPFSS hybrid model.*

Communication complexity. The $\Pi_{\text{FixedVOLE}}$ consists of the MPFSS protocol execution and the FHE-based secure computation for $\mathbf{e}_i^0, \mathbf{w}_i$, which take $\mathcal{O}(tm \log d)$ and $\mathcal{O}(km)$ communication costs, respectively. Totally, the protocol $\Pi_{\text{FixedVOLE}}$ takes $\mathcal{O}(tm \log d + km)$ in communication complexity.

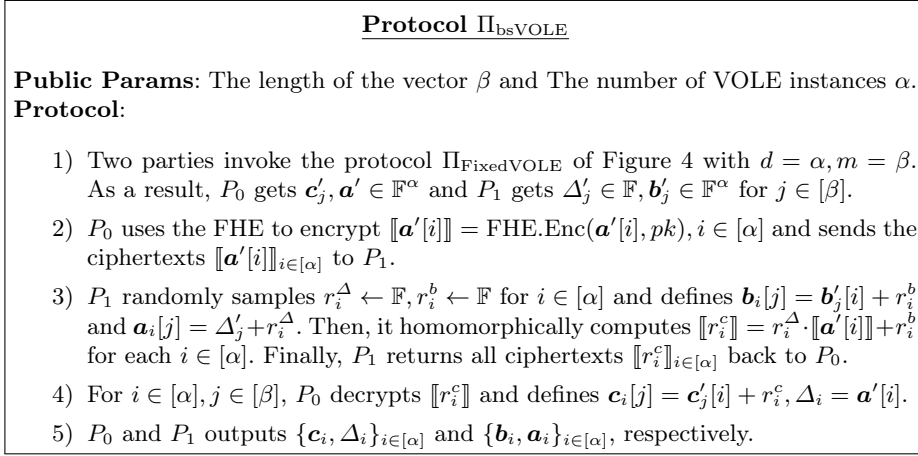


Fig. 5. The protocol Π_{bsVOLE} of the batched short VOLE

3.2 Construction of bsVOLE

Building on the $\Pi_{\text{FixedVOLE}}$ protocol, we propose the bsVOLE scheme for efficiently generating a large number of short VOLE tuples. Specifically, we first execute $\Pi_{\text{FixedVOLE}}$ to obtain ω VOLE tuples of length n , each sharing a common input vector. We then perform a lightweight remasking step using n additional OLE tuples to convert these into n fully independent VOLE tuples of length ω . This approach significantly reduces the overhead compared to directly generating short and independent VOLE tuples individually.

Concretely, first, we perform the protocol $\Pi_{\text{FixedVOLE}}$ to generate ω VOLE tuples with length n . As a result, we get $\mathbf{c}'_j = \Delta'_j \cdot \mathbf{a}' + \mathbf{b}'_j, j \in [\omega]$, wherein P_0 gets $\mathbf{a}', \mathbf{c}'_j \in \mathbb{F}^n$ and P_1 gets $\Delta'_j \in \mathbb{F}, \mathbf{b}'_j \in \mathbb{F}^n$. Second, we resize the above tuples into n VOLE tuples with the length of ω by coupling with n OLE tuples. The FHE-based OLE generation [18] is performed to get n OLE tuples $r_i^c = r_i^\Delta \cdot \mathbf{a}'[i] + r_i^b, i \in [n]$, where P_0 holds r_i^c and P_1 holds r_i^Δ, r_i^b . Then, for $i \in [n], j \in [\omega]$, P_0 defines $\mathbf{c}_i[j] = \mathbf{c}'_j[i] + r_i^c$ and $\Delta_i = \mathbf{a}'[i]$, while P_1 sets $\mathbf{b}_i[j] = \mathbf{b}'_j[i] + r_i^b$ and $\mathbf{a}_i[j] = r_i^\Delta + \Delta'_j$. After that, the tuples are changed into $\mathbf{c}_i = \Delta_i \cdot \mathbf{a}_i + \mathbf{b}_i, i \in [n]$, wherein P_0 gets $\mathbf{c}_i \in \mathbb{F}^\omega, \Delta_i \in \mathbb{F}$ and P_1 gets $\mathbf{a}_i, \mathbf{b}_i \in \mathbb{F}^\omega$.

The formal description of the bsVOLE protocol is shown in Figure 5. Theorem 2 shows the correctness and security of the protocol Π_{bsVOLE} and the full proof is given in Appendix A.2.

Theorem 2 (Security). *If the LPN assumption holds and the FHE scheme is IND-CPA secure, the protocol Π_{bsVOLE} of Figure 5 is correct and secure against semi-honest adversaries in the random oracle, fixed VOLE hybrid model.*

In certain applications, such as PSI [18], it is required that all VOLE tuples satisfy $\mathbf{a}_i[j] \neq 0$ for all $i \in [n], j \in [\omega]$. To fulfill this requirement, a minor modification to the protocol is sufficient. Recall that $\mathbf{a}_i[j] = r_i^\Delta + \Delta_j$, where r_i^Δ

is randomly chosen during OLE generation. We instruct P_1 to sample r_i^Δ such that $r_i^\Delta + \Delta_j \neq 0$ for all $j \in [\omega]$. Since r_i^Δ is uniformly sampled from $\mathbb{F}$, the result r_i^c of the OLE tuples is still uniformly distributed in $\mathbb{F}$. Therefore, the bsVOLE remains secure with the modification.

Communication complexity. The $\Pi_{\text{FixedVOLE}}$ protocol execution and the FHE-based secure computation take $\mathcal{O}(\omega t \log n + \omega k)$ and $\mathcal{O}(n)$ communication complexity, respectively. Totally, the protocol Π_{bsVOLE} takes $\mathcal{O}(t\omega \log n + k\omega + n)$. Following previous works [18, 25], we set $n = \mathcal{O}(n)$ and $\omega = \mathcal{O}(\log n / \log \log n)$. Together with the observation that t, k can be chosen in $\mathcal{O}(\sqrt{n})$ [4, 31] in this setting, the communication complexity of Π_{bsVOLE} is simplified to $\mathcal{O}(n)$.

4 Batched PMT for Online Phase

In this section, we present an efficient bPMT protocol from pre-computed VOLE tuples. We first introduce the VOLE-based PMT protocol in Section 4.1. Then, in Section 4.2, we extend the PMT protocol to the bPMT by employing CRT.

4.1 PMT from VOLE

The PMT is a two-party protocol between P_0 holding an element x and P_1 holding a set $Y = (y_1, \dots, y_n)$. After the execution, P_0 learns an indication bit output e such that $e = \text{true}$ if and only if $x \in Y$ but without knowing Y , while P_1 learns nothing. Consequently, the PSI protocol can be obtained by performing PMT for each $x \in X$ with set Y .

We assume that the correct VOLE tuple has been generated between two parties. Concretely, P_1 holds the vectors $\mathbf{a} \in \{\mathbb{F}^*\}^n, \mathbf{b} \in \mathbb{F}^n$, and P_0 obtains an integer $\Delta \in \mathbb{F}$ and a vector $\mathbf{c}$, which satisfies $\mathbf{c} = \Delta \cdot \mathbf{a} + \mathbf{b}$. The full protocol Π_{PMT} of our VOLE-based PMT is shown in Figure 6. Theorem 3 captures the correctness and security of Π_{PMT} , and the full proof is given in Appendix A.3.

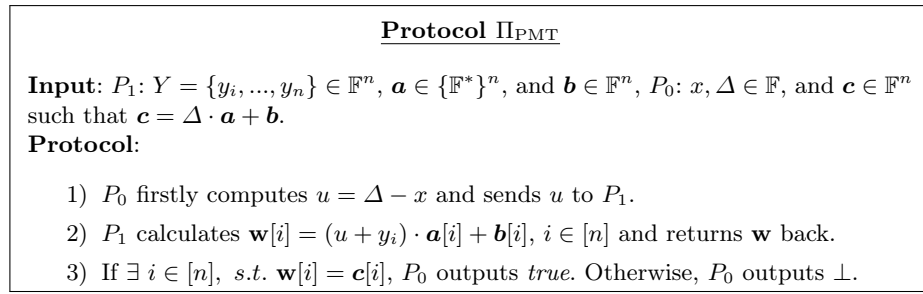


Fig. 6. The PMT protocol Π_{PMT} from VOLE

Theorem 3. *With the pre-computed VOLE tuples, the protocol Π_{PMT} of Figure 6 is correct and secure against the semi-honest adversaries in the random oracle, VOLE hybrid model.*

Furthermore, when handling an arbitrary input length ℓ , a cryptographic hash function $F : \{0, 1\}^\ell \rightarrow \mathbb{F}$ is required to map the items into $\mathbb{F}$.

The full-fledged PSI protocol can be obtained by repeatedly performing the Π_{PMT} for each $x \in X$ with set Y . Following the previous work [25, 18], we use the hashing technique to reduce the protocol complexity. Concretely, P_0 performs the cuckoo hashing with h hash functions $H_1, \dots, H_h$ and hashes all items $x \in X$ into α bins. After hashing, P_0 needs to pad empty bins with dummy items to ensure that the number of items per bin is always one. Since there is no way for P_1 to know which one of the hash functions P_0 eventually ended up using for the items in the intersection, P_1 must insert each of its items into a two-dimensional hash table using all hash functions $H_1, \dots, H_h$. According to the analysis of [25, 18], this operation increases the maximum load of a bin on the P_1 side to $\beta = \mathcal{O}(h \cdot \log(n) / \log \log(n))$. Moreover, P_1 must pad all bins to β items to avoid information leakage about its own set. Besides, we also employ the permutation-based hashing [25] to reduce the communication cost.

Using the hashing technique, P_0 only needs to compare each item x in the α hash bins with the corresponding items $y_1, \dots, y_\beta$ stored in the same bin location of the simple hash table. To perform the Π_{PMT} protocol for each bin, the two parties consume one VOLE tuple of length β . Therefore, a total of α VOLE tuples are required across all bins. These tuples are pre-generated in the offline phase using the bsVOLE protocol described in Section 3.

4.2 Batched PMT

Even with the hashing techniques, P_1 still needs to perform $\beta = \mathcal{O}(h \cdot \log n / \log \log n)$ times operations to compute $\mathbf{w} = [\mathbf{w}[1], \dots, \mathbf{w}[\beta]]$ in the Π_{PMT} for each hash bin. Instead, it would be nice if P_1 could fold the β operations into one. We achieve this by CRT-based batching techniques, a well-known and powerful technique to enable SIMD (single instruction, multiple data) operations.

For simplicity, we now focus on one bin and write the cuckoo hash bin item by x and the simple hash bin item by $\{y_1, \dots, y_\beta\}$. Employing the CRT-based batching, P_1 packs β items $(y_j, \mathbf{a}[j], \mathbf{b}[j])_{j \in [\beta]}$ into large numbers (Y^P, A^P, B^P) , respectively. Then the computation of $\mathbf{w}[j] = (u + y_j) \cdot \mathbf{a}[j] + \mathbf{b}[j]$, $j \in [\beta]$ is transformed into $W^P = (u + Y^P) \cdot A^P + B^P$, which is performed once per bin. The batching technique allows P_1 to operate on β items simultaneously, resulting in β -fold improvement in the computation. However, the multiplication computation $Y^P \cdot A^P$ is unsupported by the CRT-based batching method, posing an obstacle to protocol construction.

Note that $y_j, \mathbf{a}[j], j \in [\beta]$ are owned by P_1 , who could locally compute $t_j = y_j \cdot \mathbf{a}[j]$, $j \in [\beta]$ and pack the results $(t_1, \dots, t_\beta)$ in advance. Then, the W_P can be correctly computed by $W^P = u \cdot A^P + T^P + B^P$, which only involves addition and scale multiplication. Finally, P_0 receives the W^P and applies Batch.Unpack algorithm to get $\mathbf{w}$ and execute the subsequent protocol.

Communication optimization. CRT-based batching reduces computational overhead at the cost of slight communication growth, averaging β bits per bin.

Table 1. Comparison of β with the number of primes contained in $[h \cdot 2^{\sigma_1}, 2^{\sigma_1 + \lceil \log(h) \rceil}]$. Each item is a $\ell = 32$ -bit integer. The number of hash functions is $h = 3$. $\alpha = 1.27 \cdot n$.

Set size n	2^{20}	2^{22}	2^{24}	2^{26}
β	28	28	29	29
The number of primes	431	125	37	11

To ensure the correctness of batching, all CRT prime moduli should be greater than the corresponding item value. After applying permutation-based hashing, the length of the items stored in the hash table is $\sigma_1 + \lceil \log(h) \rceil$ bits, where $\sigma_1 = \ell - \lfloor \log(\alpha) \rfloor$. Therefore, all prime moduli should satisfy $q_i \geq 2^{\sigma_1 + \lceil \log(h) \rceil}$ ($i \in [\beta]$). It means each prime moduli of $(q_1, \dots, q_\beta)$ at least be $\sigma_1 + \lceil \log(h) \rceil + 1$ bits, and the computation result W^P is $\beta(\sigma_1 + \lceil \log(h) \rceil + 1)$ bits, which is β bits larger than directly transferring $\mathbf{w}$ in the naive PMT protocol without batching.

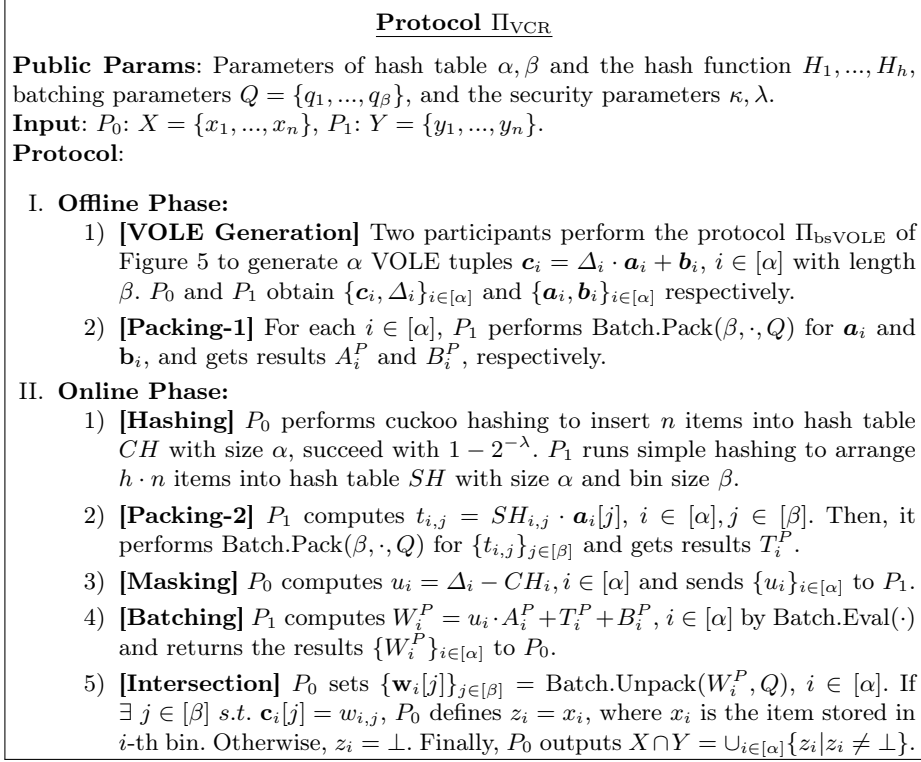
We eliminate the communication growth by optimizing the selection of primes for batching. As shown in [25, 27, 18], with the permutation-based hashing, each item in a hash bin falls within the range $[0, h \cdot 2^{\sigma_1}]$. Therefore, any prime in the interval $[h \cdot 2^{\sigma_1}, 2^{\sigma_1 + \lceil \log_2(h) \rceil}]$ satisfies the requirements for CRT batching. This implies that as long as the interval $[h \cdot 2^{\sigma_1}, 2^{\sigma_1 + \lceil \log_2(h) \rceil}]$ contains more than β distinct primes, sufficient batching capacity is ensured, and communication overhead remains constant regardless of the set size.

Referring to the analysis of [18, 27], the setting of $h = 3, \alpha = 1.27 \cdot n$ results in the optimal cost of online communication for set sizes between 2^{20} and 2^{26} . Therefore, we mainly analyze the parameter β , which represents the number of primes required for CRT-based batching, and examine the number of primes in the case of $h = 3, \alpha = 1.27 \cdot n$ with various set sizes. As shown in Table 1, the number of primes within the interval $[h \cdot 2^{\sigma_1}, 2^{\sigma_1 + \lceil \log(h) \rceil}]$ exceeds β in most cases, except for $n = 2^{26}$. Consequently, by optimizing the selection of prime numbers, the aforementioned communication growth can be effectively mitigated in the majority of scenarios. For the specific case $n = 2^{26}$, the β items can be partitioned into multiple segments for batching to minimize communication overhead. For instance, $\beta = 29$ items can be divided into three segments containing 10, 10, and 9 items, respectively. This approach reduces the number of required primes to 10 per bin (less than the primes in $[h \cdot 2^{\sigma_1}, 2^{\sigma_1 + \lceil \log(h) \rceil}]$), allowing batching for $n = 2^{26}$ without incurring additional communication overhead.

5 Full Protocol of VCR and Security

We have designed a novel bsVOLE protocol to improve the offline efficiency and proposed bPMT to accelerate the online computation. In this section, we integrate these two building blocks to construct our efficient PSI protocol, VCR.

As shown in Figure 7, VCR is designed in the offline-online paradigm. In the offline phase, two parties, i.e., P_0 and P_1 , invoke the bsVOLE to generate the VOLE tuples. For the online phase, two parties first perform the hashing to

Fig. 7. Full protocol Π_{VCR} of VCR

insert their items into the hash tables. Then, they invoke the bPMT protocol for each bin of hash tables to get the intersection. In particular, since all the vectors $\mathbf{a}_i, \mathbf{b}_i, i \in [\alpha]$ have been gotten in the offline phase, P_1 could pack them in the offline phase to save the online computational cost. Theorem 4 captures the security of VCR, and the full proof is given in Appendix A.4.

Theorem 4 (Security). *If hash functions are collision-resistant, the protocol Π_{VCR} of Figure 7 is secure against semi-honest adversaries in the random oracle, bsVOLE hybrid model.*

6 Evaluation

Testbeds. We implemented VCR in C++ and benchmarked it on an Intel(R) Xeon(R) Silver 4314 CPU with 2.40 GHz and a 64GB RAM machine. We perform the experiments in different network settings, including 5 Gbps (LAN), 1 Gbps, 100 Mbps, and 10 Mbps, each with 40 ms round-trip time (RTT). The network bandwidth and latency are simulated using the Linux traffic control (tc) tools. All experiments are executed in a single thread unless otherwise noted.

Parameters. Following previous work [18], the set size $n \in \{2^{20}, 2^{22}, 2^{24}, 2^{26}\}$ and item bit length is of $\ell = 32$, which is sufficient for several applications, such

Table 2. Offline comparison between VCR and OT- and FHE-based methods of [18]. DNF: Did not finish in 15 minutes. Best protocol within a setting is marked in bold.

n		2^{20}			2^{22}			2^{24}			2^{26}		
Method		[18]		VCR	[18]		VCR	[18]		VCR	[18]		VCR
		FHE	OT	bsVOLE	FHE	OT	bsVOLE	FHE	OT	bsVOLE	FHE	OT	bsVOLE
Time (s)	5 Gbps	14.7	44.7	6.6	58.6	150.4	24.1	240.3	519.9	93.7	DNF	DNF	445.6
	1 Gbps	18.1	205.7	7.5	71.6	703.0	26.6	289.8	DNF	101.5	DNF	DNF	470.5
	100 Mbps	55.6	DNF	9.8	221.6	DNF	32.2	DNF	DNF	129.8	DNF	DNF	593.6
	10 Mbps	430.3	DNF	30.8	DNF	DNF	93.8	DNF	DNF	345.7	DNF	DNF	DNF
Communication (MB)		482	23896	30.3	1915	81930	81.8	7951	282855	288.6	DNF	905135	1167

as private contact discovery [17, 14] or blacklist matching [27, 8]. Longer domains can be mapped to 32 bits without collisions using a public dictionary. According to theoretical analysis in [18, Section V], we set $h = 3, \alpha = 1.27n$ for the lowest online communication cost. We set $\lambda = 40$ bits and $\kappa = 128$ bits.

6.1 Offline Phase

In this subsection, we compare our offline phase with the purely cryptographic (FHE- and OT-based) offline phase of [18], using their publicly available implementation. Our offline VOLE generation is evaluated using the following configurations: 1) For MPFSS, we adopt the library from [31], incorporating the optimization proposed in [29]; 2) For FHE, we use the BFV cryptosystem [11] from Microsoft’s SEAL library [32]. Specifically, we set the polynomial modulus degree to 8192 and the coefficient modulus bit length to 218, which matches the parameters used in the FHE-based offline phase of [18].

Table 2 demonstrates the superiority of our offline phase. Compared to the OT and FHE-based offline phase of [18], we observe a speedup of $14.5\times$ and $4.6\times$ for our offline phase on average while reducing the communication costs on average of $886.5\times$ and $22.3\times$, respectively. The performance improvement stems mainly from our bsVOLE protocol. Our bsVOLE only takes $\mathcal{O}(n)$ homomorphic operations and a highly efficient LPN-based long VOLE generation. In contrast, the FHE-based (resp. OT-based) OLE generation of prior work [18] incurs a complexity of $\mathcal{O}(n \log(n))$ (resp. $\mathcal{O}(n\ell \log n)$) for FHE operations (resp. OT instances). This also explains why the bsVOLE protocol in VCR significantly reduces the offline communication compared to the purely cryptographic (OT- and FHE-based) offline phase of [18]. Low communication overhead makes the offline of VCR insensitive to bandwidth limitations. For example, when $n = 2^{20}$ and the bandwidth is 10 Mbps, VCR can execute the offline in 30.8 seconds, which is at least $13.9\times$ faster than the purely cryptographic-based offline of [18]. Besides, VCR also performs particularly well when handling large-scale inputs. When $n = 2^{26}$, our method completes in 445.6 seconds at 5 Gbps, whereas OT- and FHE-based offline phases of [18] both fail to finish within 15 minutes.

6.2 Online Phase

We benchmark the online phase of VCR and compare its performance with several SOTA protocols [18, 28, 9, 20], using their publicly available implementa-

Table 3. Comparison between VCR and previous works [20, 9, 28, 18]. DNF: Did not finish in 15 minutes. Online: Online phase cost. Total: Online and offline phases cost. The best protocol within a setting is marked in bold.

n	Time and Communication	VCR (ours)		[18]		[28]		[9]	[20]
		Online	Total	Online	Total	Online	Total	Online	Online
2^{20}	5 Gbps	0.2	6.8	0.4	15.1	1.1	1.3	8.5	3.9
	1 Gbps	0.6	8.1	0.8	18.9	1.4	1.7	8.7	4.2
	100 Mbps	5.3	15.1	5.6	61.2	3.7	4.1	10.5	16.5
	10 Mbps	52.5	83.3	52.9	483.2	28.6	29.2	77.1	117.9
	Communication (MB)	64	94.3	64	546	33	34	89	114
2^{22}	5 Gbps	0.8	24.9	1.5	59.49	4.6	5.1	32.6	16.8
	1 Gbps	2.3	28.9	3	74.2	5.1	5.8	33.8	17.8
	100 Mbps	18.9	51.1	19.6	241.2	15.2	15.9	38.6	62.4
	10 Mbps	187.4	281.2	188.2	DNF	114.5	115.5	305.3	492.2
	Communication (MB)	221	302.8	221	2136	132	133	358	471
2^{24}	5 Gbps	2.4	96.1	6.2	243.4	22.6	25	154.6	62.7
	1 Gbps	7.8	109.3	11.4	301.2	23.3	25.7	157.8	66.8
	100 Mbps	63.5	193.3	67.6	DNF	64.5	67.2	176.5	228.6
	10 Mbps	656.6	DNF	662.7	DNF	461.2	464.5	DNF	DNF
	Communication (MB)	762	1050.6	762	8713	526	527	1442	1894
2^{26}	5 Gbps	10.5	456.1	19.2	DNF	86.6	97.6	710.6	374.2
	1 Gbps	27.9	498.4	36.1	DNF	102.2	113.6	722.3	389.9
	100 Mbps	207.5	801.1	218.1	DNF	266.8	279.6	895.4	DNF
	10 Mbps	DNF	DNF	DNF	DNF	DNF	DNF	DNF	DNF
	Communication (MB)	2438	3605	2438	DNF	2776	2777	5769	7800

tions. The experimental results are summarized in Table 3, with total cost also reported for reference. For the work of [28], we evaluate the “SD” variant due to its communication-efficient optimization. As the protocols in [9] and [20] are executed entirely during the online phase, we report only their online costs.

Compared to Kerschbaum *et al.* [18]. We first compare VCR with the work of Kerschbaum *et al.* [18], which is the most online efficient PSI protocol. Table 3 demonstrates the superiority of VCR in the online phase, particularly in terms of running time across different network settings and dataset sizes. In the LAN, our online phase is $1.9\text{--}2.6\times$ faster than the protocol of [18]. We attribute this improvement to our bPMT protocol, which uses optimized CRT-based batching to reduce the online running time. More importantly, VCR realizes these improvements without communication growth. That is also the reason why VCR still outperforms previous work [18] in 1 Gbps, 100 Mbps, and 10 Mbps. Concretely, VCR improves the online running time by approximately $1.4\times$ (resp. $1.1\times$) in the 1 Gbps (resp. 100 Mbps) network setting compared to [18].

Compared to Raghuraman and Rindal [28]: The protocol of Raghuraman and Rindal [28] is an optimal variant of [30] and outperforms prior art in terms of the communication cost. Compared to [28], the online phase of VCR is $5.5\text{--}9.4\times$ faster in LAN. This improvement mainly stems from the efficient online bPMT protocol, which only involves fast plaintext field operation. Since the protocol of [28] has lower communication complexity ($\mathcal{O}(n)$) compared to VCR ($\mathcal{O}(n \log n / \log \log n)$) in online, the work of [28] has the lower communication cost and performs slightly better than VCR in low-performance network environ-

ments when $n = \{2^{20}, 2^{22}, 2^{24}\}$. However, VCR is more efficient for large-scale sets. For the case $n = 2^{26}$, VCR saves 13% communication cost and improves the running time by approximately $1.3\times$ compared to [28] in the 100 Mbps setting.

Compared to Chase and Miao [9]: The work of Chase and Miao [9] is more efficient in moderate bandwidth networks (e.g., 30-100 Mbps), while VCR excels particularly in LAN. Therefore, we compare VCR with it in both LAN and 100 Mbps. Our online phase is $42.5\text{-}64.4\times$ (resp. $2.0\text{-}4.3\times$) faster than [9] in LAN (resp. 100 Mbps) and requires less communication.

Compared to Kolesnikov *et al.* [20]: The work of Kolesnikov *et al.* [20] is suitable for high-performance network environments since it is particularly fast but comes with a comparatively large communication overhead. We mainly compare VCR with it in LAN. As is shown in Table 2, the online phase of VCR is $19.5\text{-}35.6\times$ faster than [20] and saves 43.8% – 68.7% communication cost.

7 Related works

PSI dates back to the 1980s [21] and was initially based on the Diffie-Hellman (DH) protocol. The main idea is that both parties construct the oblivious pseudorandom function (OPRF) protocol through the DH protocol and obtain the intersection result by comparing the OPRF results of the inputs. Many modern PSI protocols [15, 7] are still based on it for low communication cost and constant round complexity. However, these protocols always come at the cost of high computational overhead.

In the past several years, PSI has been extensively studied, and many efficient protocols have been proposed. Researchers take advantage of the OT extension, which was proposed by Ishai *et al.* [16], to construct more computationally efficient PSI protocols [26] with many derivatives [25, 20, 27, 23, 9] at the expense of increased communication cost.

The works of Garimella *et al.* [12] and Pinkas *et al.* [24] addressed the limitations of previous works by introducing a new structure called oblivious key-value store (OKVS) to PSI. They obtained a more efficient PSI protocol than the previous. Later, Rindal and Schoppmann [30] combined the OKVS with VOLE to improve efficiency and became the most efficient PSI protocol for communication at the time. However, the non-OKVS-based protocol of Kolesnikov *et al.* [20] remains the most computationally efficient one. Shortly after, Raghuraman *et al.* [28] propose an improved OKVS construction and use it, along with subfield-VOLE, as a building block to obtain the PSI protocol that is more efficient than [20] and [30] in both communication and computation.

Surprisingly, Kerschbaum *et al.* [18] directly used the equality relationship satisfied by OLE tuples to construct a comparison protocol. Based on this, they implemented a PSI protocol that is the most computationally efficient and free of cryptographic operations in the online phase.

8 Conclusion

In this paper, we proposed VCR, an efficient PSI protocol from VOLE. First, we propose the bPMT protocol for the online phase to compute the intersection from pre-computed VOLE. Then, we propose a new approach, bsVOLE, for efficient offline VOLE generation. Experiments show that VCR outperforms the currently best-related works [20, 9, 28, 18]. Moreover, bsVOLE offers a new perspective on efficiently generating large-scale, short-length VOLE tuples. For future work, we are willing to extend VCR against malicious adversaries.

References

1. Applebaum, B., Damgård, I., Ishai, Y., Nielsen, M., Zichron, L.: Secure arithmetic computation with constant computational overhead. In: Annual International Cryptology Conference. pp. 223–254. Springer (2017)
2. Arbitman, Y., Naor, M., Segev, G.: Backyard cuckoo hashing: Constant worst-case operations with a succinct representation. In: 2010 IEEE 51st Annual symposium on foundations of computer science. pp. 787–796. IEEE (2010)
3. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y.: Compressing vector ole. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. pp. 896–912 (2018)
4. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Rindal, P., Scholl, P.: Efficient two-round ot extension and silent non-interactive secure computation. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 291–308 (2019)
5. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 337–367. Springer (2015)
6. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing: Improvements and extensions. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 1292–1303 (2016)
7. Buddhavarapu, P., Knox, A., Mohassel, P., Sengupta, S., Taubeneck, E., Vlaskin, V.: Private matching for compute. Cryptology ePrint Archive (2020)
8. Bursztein, E., Hamburg, M., Lagarenne, J., Boneh, D.: Openconflict: Preventing real time map hacks in online games. In: 2011 IEEE Symposium on Security and Privacy. pp. 506–520. IEEE (2011)
9. Chase, M., Miao, P.: Private set intersection in the internet setting from lightweight oblivious prf. In: Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III 40. pp. 34–63. Springer (2020)
10. Chen, Y., Zhang, M., Zhang, C., Dong, M., Liu, W.: Private set operations from multi-query reverse private membership test. In: IACR International Conference on Public-Key Cryptography. pp. 387–416. Springer (2024)
11. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive (2012)
12. Garimella, G., Pinkas, B., Rosulek, M., Trieu, N., Yanai, A.: Oblivious key-value stores and amplification for private set intersection. In: Advances in Cryptology–CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Proceedings, Part II 41. pp. 395–425. Springer (2021)

13. Ghosh, S., Simkin, M.: The communication complexity of threshold private set intersection. In: Annual International Cryptology Conference. pp. 3–29. Springer (2019)
14. Hetz, L., Schneider, T., Weinert, C.: Scaling mobile private contact discovery to billions of users. Cryptology ePrint Archive (2023)
15. Ion, M., Kreuter, B., Nergiz, A.E., Patel, S., Saxena, S., Seth, K., Raykova, M., Shanahan, D., Yung, M.: On deploying secure computing: Private intersection-sum-with-cardinality. In: 2020 IEEE European Symposium on Security and Privacy (EuroS&P). pp. 370–389. IEEE (2020)
16. Ishai, Y., Kilian, J., Nissim, K., Petrank, E.: Extending oblivious transfers efficiently. In: Annual International Cryptology Conference. pp. 145–161. Springer (2003)
17. Kales, D., Rechberger, C., Schneider, T., Senker, M., Weinert, C.: Mobile private contact discovery at scale. In: 28th USENIX Security Symposium (USENIX Security 19). pp. 1447–1464 (2019)
18. Kerschbaum, F., Blass, E.O., Mahdavi, R.A.: Faster secure comparisons with offline phase for efficient private set intersection. In: Network and Distributed System Security Symposium (NDSS) 2023 (2023)
19. Kiss, Á., Liu, J., Schneider, T., Asokan, N., Pinkas, B.: Private set intersection for unequal set sizes with mobile applications. Proc. Priv. Enhancing Technol. (2017.4), 177–197 (2017)
20. Kolesnikov, V., Kumaresan, R., Rosulek, M., Trieu, N.: Efficient batched oblivious prf with applications to private set intersection. In: 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 818–829 (2016)
21. Meadows, C.: A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party. In: 1986 IEEE Symposium on Security and Privacy. pp. 134–134. IEEE (1986)
22. Pagh, R., Rodler, F.F.: Cuckoo hashing. Journal of Algorithms **51**(2), 122–144 (2004)
23. Pinkas, B., Rosulek, M., Trieu, N., Yanai, A.: Spot-light: lightweight private set intersection from sparse ot extension. In: Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39. pp. 401–431. Springer (2019)
24. Pinkas, B., Rosulek, M., Trieu, N., Yanai, A.: Psi from paxos: fast, malicious private set intersection. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 739–767. Springer (2020)
25. Pinkas, B., Schneider, T., Segev, G., Zohner, M.: Phasing: Private set intersection using permutation-based hashing. In: 24th USENIX Security Symposium (USENIX Security 15). pp. 515–530 (2015)
26. Pinkas, B., Schneider, T., Zohner, M.: Faster private set intersection based on ot extension. In: 23rd USENIX Security Symposium (USENIX Security 14). pp. 797–812 (2014)
27. Pinkas, B., Schneider, T., Zohner, M.: Scalable private set intersection based on ot extension. ACM Transactions on Privacy and Security (TOPS) **21**(2), 1–35 (2018)
28. Raghuraman, S., Rindal, P.: Blazing fast psi from improved okvs and subfield vole. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 2505–2517 (2022)
29. Raghuraman, S., Rindal, P., Tanguy, T.: Expand-convolute codes for pseudorandom correlation generators from lpn. In: Annual International Cryptology Conference. pp. 602–632. Springer (2023)

30. Rindal, P., Schoppmann, P.: Vole-psi: fast oprf and circuit-psi from vector-ole. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 901–930. Springer (2021)
31. Schoppmann, P., Gascón, A., Reichert, L., Raykova, M.: Distributed vector-ole: Improved constructions and implementation. In: ACM SIGSAC Conference on Computer and Communications Security. pp. 1055–1072 (2019)
32. Microsoft SEAL (release 4.0). <https://github.com/Microsoft/SEAL> (Mar 2022), microsoft Research, Redmond, WA.
33. Weng, C., Yang, K., Katz, J., Wang, X.: Wolverine: fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In: 2021 IEEE Symposium on Security and Privacy. pp. 1074–1091. IEEE (2021)

Appendix A Theorem Proof

A.1 Proof of Theorem 1

Proof. First, we state the correctness. According to the definition of MPFSS protocol [31, 4], we get $\Delta_i \cdot \mu = \mu_i^0 + \mu_i^1$ in protocol $\Pi_{\text{FixedVOLE}}$. Therefore, for each $i \in [m]$, the protocol $\Pi_{\text{FixedVOLE}}$ holds that

$$\begin{aligned} \Delta'_i \cdot \mathbf{a}' + \mathbf{b}'_i &= \Delta'_i(\mathbf{v} \cdot \mathcal{C} + \mu) + \mathbf{u}_i \cdot \mathcal{C} - \mu_i^1 \\ &= \mathbf{w}_i \cdot \mathcal{C} + \mu_i^0 = \mathbf{c}'_i \end{aligned}$$

To show the security, we need to show that the two security properties from Definition 1 hold. We prove the first one and give the simulators as follows.

- **Corrupt P_1 :** $\text{Sim}_1(\text{seed}_1^i)$ simulates the view of corrupt P_1 as follows. In step 3, Sim_1 encrypts random values to simulate $\llbracket \mathbf{e} \rrbracket, \llbracket \mathbf{v} \rrbracket$. In step 5, Sim_1 simulates uniform outputs K_1^i from the MPFSS protocol. The view of the underlying MPFSS simulator is indistinguishable. Since the FHE scheme is IND-CPA secure, the simulation is indistinguishable from the real view.
- **Corrupt P_0 :** $\text{Sim}_0(\text{seed}_0^i)$ simulates the view of corrupt P_0 as follows. In step 4, Sim_0 encrypts random vectors to simulate $\llbracket \mathbf{e}_i^0 \rrbracket$ and $\llbracket \mathbf{w}_i \rrbracket$. In step 5, Sim_0 simulates uniform outputs K_0^i from the MPFSS protocol. The view of the underlying MPFSS simulator is indistinguishable. Since the FHE scheme is IND-CPA secure, the simulation is indistinguishable from the real view.

To prove the second property, we show a transition between the $(\mathbf{c}'_i, \mathbf{a}', \text{seed}_1^i)$ and $(\mathbf{c}_i^*, \mathbf{a}^*, \text{seed}_1^i)$ through a sequence of steps. In the first step, we compute $(\text{seed}_0^i, \text{seed}_1^i)$ from $\text{Setup}(1^\lambda, \mathbb{F}, d, n)$, set $(\mathbf{c}'_i, \mathbf{a}')$ from $\text{Expand}(i, 0, \text{seed}_0^i)$, and send $((\mathbf{c}'_i, \mathbf{a}', \text{seed}_1^i))$ to adversary $\mathcal{A}$. Then, we replace the K_1^i with the simulated MPFSS key, which is generated from $\mathbb{F}$ and n alone. The security of MPFSS guarantees that this simulated key is indistinguishable from the real one. In the third step, we consider the distribution $\mathbf{c}_i$ and $\mathbf{a}$. Note that $\mathbf{a}' = \mathbf{v} \cdot \mathcal{C} + \mu$ and $\mathbf{c}'_i = \Delta'_i \cdot \mathbf{a}' + \mathbf{b}'_i = \Delta_i \cdot \mathbf{a}' + \mathbf{u}_i \cdot \mathcal{C} - \mu_i^1$. We replace the $\mathbf{c}'_i$ and $\mathbf{a}'$ with $\mathbf{a}^* \leftarrow \mathbf{F}^d$ and $\mathbf{c}_i^* = \Delta_i \cdot \mathbf{a}^* + \mathbf{u}_i \cdot \mathcal{C} - \mu_i^1$, respectively. By the LPN assumption, $\mathbf{a}'$ and $\mathbf{a}^*$ are indistinguishable. Since the $\mathbf{c}'_i$ and $\mathbf{c}_i^*$ are computed in the same way, the change in the third step is also indistinguishable for $\mathcal{A}$.

A.2 Proof of Theorem 2

Proof. First, we state the correctness of Π_{bsVOLE} . For $i \in [\alpha], j \in [\beta]$, it holds

$$\begin{aligned}\Delta_i \cdot \mathbf{a}_i[j] + \mathbf{b}_i[j] &= \mathbf{a}'[i] \cdot (r_i^\Delta + \Delta'_j) + \mathbf{b}'_j[i] + r_i^b \\ &= \Delta'_j \cdot \mathbf{a}'[i] + \mathbf{b}'_j[i] + r_i^\Delta \cdot \mathbf{a}'[i] + r_i^b \\ &= \mathbf{c}'_j[i] + r_i^c = \mathbf{c}_i[j]\end{aligned}$$

Then, we prove the security and give the simulators Sim_0 and Sim_1 as follows.

- **Corrupt P_0 .** Sim_0 invokes $\mathcal{F}_{\text{FixedVOLE}}$ and appends the output to the view. Since the r_i^Δ and r_i^b ($i \in [\alpha]$) are randomly sampled by P_1 , the result of r_i^c is pseudorandom for P_0 . Therefore, Sim_0 encrypts random numbers to simulate the ciphertexts in step 3. The security of the protocol $\Pi_{\text{FixedVOLE}}$ guarantees the view in simulation is indistinguishable from the real view.
- **Corrupt P_1 .** Sim_1 invokes $\mathcal{F}_{\text{FixedVOLE}}$ and appends the output to the view. Then it chooses random numbers and encrypts them to simulate the ciphertexts in step 2. The security of the FHE scheme and the protocol $\Pi_{\text{FixedVOLE}}$ guarantee the view in simulation is indistinguishable from the real view.

A.3 Proof of Theorem 3

Proof. First, we state the correctness of protocol Π_{PMT} as follows: Consider the message $\mathbf{w}[i]$, $i \in [n]$, obtained by P_0 , $\mathbf{w}[i]$ can be detailed as $\mathbf{w}[i] = (\Delta - x + y_i) \cdot \mathbf{a}[i] + \mathbf{b}[i]$. Since the VOLE tuples such that $\mathbf{c} = \Delta \cdot \mathbf{a} + \mathbf{b}$, we have

$$\begin{aligned}\mathbf{w}[i] - \mathbf{c}[i] &= (\Delta - x + y_i) \cdot \mathbf{a}[i] + \mathbf{b}[i] - \mathbf{c}[i] \\ &= (y_i - x) \cdot \mathbf{a}[i]\end{aligned}$$

With the condition $\mathbf{a}[i] \in \mathbb{F}^* \neq 0, i \in [n]$, the $\mathbf{w}[i] - \mathbf{c}[i]$ is equivalent to $y_i = x$. Therefore, we conclude if $\exists i \in [n]$ s.t. $\mathbf{w}[i] = \mathbf{c}[i]$, $x \in Y$.

Then, we prove the security and give the simulators Sim_0 and Sim_1 as follows.

- **Corrupt P_1 :** Since x is unknown to P_1 and Δ is uniformly distributed in $\mathbb{F}$ for P_1 according to the definition of VOLE, Sim_1 sample random numbers to simulate u . The security of VOLE guarantees the view in simulation is indistinguishable from the real view.
- **Corrupt P_0 :** If $x \in Y$, then the simulator outputs *true*. We have shown perfect correctness and hence only analyze the case $x \notin Y$. We show that $\mathbf{w}[i], i \in [n]$ is an uniformly distributed in $\mathbb{F} \setminus \{\mathbf{c}[i]\}, i \in [n]$ for P_0 . With VOLE $\mathbf{c}[i] = \Delta \cdot \mathbf{a}[i] + \mathbf{b}[i]$, $i \in [n]$, the $\mathbf{w}[i]$ can be rewritten as $\mathbf{w}[i] = (y_i - x) \cdot \mathbf{a}[i] + \mathbf{c}[i]$. The simulator needs to output $\mathbf{w}[i]$, but given $\mathbf{c}[i]$ as input. $y_i - x$ is in $\mathbb{F}^*$ and $\mathbf{a}[i]$ is uniformly distributed in $\mathbb{F}^*$. $\mathbb{F}^*$ forms the multiplicative sub-group in $\mathbb{F}$. Hence $(y_i - x) \cdot \mathbf{a}[i]$ is uniformly distributed in $\mathbb{F}^*$. Sim_0 outputs the sum of a random number in $\mathbb{F}^*$ and $\mathbf{c}[i]$ to simulate $\mathbf{w}[i]$. By the security of VOLE, this view in simulation is indistinguishable from the real view.

A.4 Proof of Theorem 4

Proof. We prove the security of the protocol Π_{VCR} and give the simulators Sim_0 and Sim_1 as follows.

- **Corrupt P_0 .** Sim_0 invokes bsVOLE and appends the output to the view. Then, it chooses random numbers to simulate the $\{W_{i,P}\}_{i \in [\alpha]}$ in step 4. The security of the protocol Π_{bsVOLE} guarantees the view in simulation is indistinguishable from the real view.
- **Corrupt P_1 .** Sim_1 invokes bsVOLE and appends the output to the view. Then, it chooses random numbers to simulate the $\{u_i\}_{i \in [\alpha]}$ in step 3. The security of the protocol Π_{bsVOLE} guarantees the view in simulation is indistinguishable from the real view.